

CryptoBroker · CryptoSim

Projekt-Handout · Abschlusspräsentation (Ende 4. Semester) · Software Engineering · TINF24B4 · DHBW Karlsruhe

Team: Lucas (Backend & DevOps) · Julian (Architektur & DB) · Jonathan (Frontend & UI) · Sara (Anforderungen & QA) · github.com/Lucas-Klein1/CryptoBroker

Projektziel & Vision

Vision: „Kryptowährungen verstehen und das Handeln üben – ohne echtes Geld zu riskieren.“ CryptoBroker bildet einen echten Krypto-Markt als risikofreie Lernumgebung nach: Live-Kurse von 100 Coins (CoinGecko), virtuelles Startkapital, persönliches Portfolio und Bestenliste. Zielgruppe: Einsteiger, Studierende, Hobby-Trader.

Statistik

112,75 h Netto-Aufwand (Aufgabenstunden, geteilte Aufgaben einmal gezählt) von 119,5 h geplant – 94 % Auslastung. Erfasst über das GitHub-Scrum-Board.

Stunden pro Disziplin: Anforderungen 5 h · Planung & Entwurf 35 h · Implementierung 61,5 h · Test & Qualitätssicherung 8,5 h · ohne Zuteilung 2,75 h (Summe 112,75 h).

Stunden pro Phase: Konzeption & Ausarbeitung (Sem. 3, Sprints 1–8) 72,75 h · Konstruktion & Abschluss (Sem. 4, Sprints 9–13) 40 h.

Personenstunden: Summe 179 h (Pro-Person-Tabelle unten); die Differenz von 66 h zum Netto-Aufwand entfällt auf gemeinsam bearbeitete Aufgaben – Pair-Programming, Reviews, Walkthrough. Scrum-Board: 54 Items, 52 abgeschlossen.

Stunden pro Person – Personenstunden & Hauptbeiträge (Ø ≈ 44,8 h):

Person	Stunden	Hauptbeiträge
Lucas	46,5 h	Backend (Flask, REST-API, app.py), CI/CD, Docker
Julian	47 h	Datenbankmodelle (SQLite), Service-Layer, Technologiewechsel
Jonathan	43 h	Frontend (HTML/CSS/JS, Chart.js, Templates), UI & Dark Theme
Sara	42,5 h	Anforderungsanalyse (SRS, ASR), Dokumentation, Blog, Testing

Demo-Highlights

- **Voller Trade-Workflow live:** Login (bcrypt) → Dashboard (100 Coins live) → Coin-Detail mit 365-Tage-Chart → Kauf/Verkauf → Portfolio → Bestenliste.
- **Drei Eingabemodi beim Trade:** nach Euro-Betrag, Menge oder Prozent des Bestands.
- **Echte Live-Kursdaten** statt Fantasiewerten; Guthaben/Bestände per Event-Sourcing live aus allen Transaktionen berechnet.

Technologiewechsel: JavaFX → Python/Flask

Start in Sprint 1 mit Java/JavaFX (Desktop). In Sprint 2 bewusster Wechsel zu Python/Flask (Web): klare Frontend/Backend-Trennung, REST, einfache Docker-Verteilung, teamvertraut. Aufwand einmalig ~7 h – Key Learning: Technologie-Risiken früh sichtbar machen; Scrum ermöglichte das flexible Reagieren.

Architektur & Architekturentscheidungen

Stil: 3-Schichten-Architektur mit MVC-Elementen im Presentation Layer (Client-Server), dokumentiert in arc42. Kernentscheidungen: zentrale database.py als einzige DB-Verbindung · Thin Controller – Fat Service (Logik im Service-Layer) · Session-basierte Authentifizierung.

Architekturentscheidung (5 ADRs, arc42 Kap. 9)	Begründung
3-Schichten statt Monolith	Trennung der Belange, isoliert testbare Logik
Flask statt Django/FastAPI	Minimaler Overhead, teamvertraut
SQLite statt PostgreSQL	Keine Server-Infrastruktur, leicht austauschbar
bcrypt für Hashing	Industrie-Standard, automatischer Salt
CoinGecko-API	Kostenlos & zuverlässig statt eigener Datenquelle

Datenbank-Design

Tabelle	Felder	Zweck
accounts	id (PK), name (unique), pw (bcrypt), created_at	Benutzerkonten
transactions	id, acc_id (FK), coin_id (FK), type (BUY/SELL), amount, price, timestamp	Handelsvorgänge
coins	id, symbol, name, current_price, market_cap, rank, image	Marktdaten-Cache
favorites	acc_id (PK/FK), coin_id (PK/FK)	Watchlist je Nutzer
<coin>_history	timestamp_ms (unique), price	Kursverlauf je Coin

Event-Sourcing: Guthaben und Bestände werden nicht gespeichert, sondern live aus allen Transaktionen berechnet.

Verwendete Tools, Plattformen & Bibliotheken

Backend	Frontend	DevOps / Infra	Methodik / Doku
Python 3	HTML5	Docker	Scrum / Agile
Flask	CSS3	GitHub Actions	SRS & ASR
Jinja2	JavaScript	GitHub (VCS)	arc42 + 5 ADRs
SQLite	Chart.js	pytest · pytest-cov	Sprint-Blog
bcrypt		radon	Review-Protokoll

Tests, Metriken & CI/CD

Tests: 35 Testfälle grün (33 Unit + 2 Integration), **92 % Coverage** auf der Service-Schicht (market_service, Ziel $\geq 80\%$). pytest + pytest-cov, isolierte In-Memory-SQLite pro Test. Routen/Templates/API-Sync bewusst ausgenommen.

Metriken (radon, automatisch in CI über 87 Blöcke): Ø zyklomatische Komplexität **A (2,68)**, Wartbarkeitsindex je Modul durchweg grün, ~960 Code-Zeilen (SLOC). execute_trade nach IOSP A (1); die 3 Eingabemodi liegen in place_order (Service), wodurch die /trade-Route nur noch B (7) hat. Bewusst belassener Hotspot: get_leaderboard C (11) – dokumentierter Tradeoff.

CI/CD (GitHub Actions, Trigger: push & PR auf main): 1) Tests (Syntaxcheck, pytest+Coverage, JUnit-Artefakte) → 2) Metriken (radon cc/mi/raw) → 3) Build & Smoke (Docker-Image, Container-Start, Smoke-Test). Stufe 2 & 3 nur nach grünen Tests.

Clean Code, Refactoring & Review

Prinzipien: SRP, DRY, IOSP, SoC, kleine Funktionen – gezielt an den Komplexitäts-Hotspots. 4+ dokumentierte Refactorings + Design Patterns: execute_trade nach IOSP entflochten; DRY bei Tabellennamen & BUY/SELL-Logik; login_or_register entflochten; @login_required-Decorator (Decorator Pattern).

Review (Walkthrough, 25.05.2026): Rollen Moderator/Vorleser/Protokoll/Zeitwächter; geprüft app.py, market_service, coin_sync_service. 5 Befunde, alle behoben – u. a. DROP TABLE beim Start → INSERT OR REPLACE; duplizierter Login-Check → @login_required-Decorator. Protokoll: docs/review-protocol.md.

Anwendungsfälle (Auszug)

Implementiert (Demo-Fokus): UC-01 Home · UC-02 Login/Logout · UC-03 Dashboard · UC-04 Coin-Übersicht · UC-05 Coin-Detail · UC-06 Account-Navigation · UC-07 Start via Docker.

Weitere / Backlog: UC-08 Registrieren · UC-09 Virtuelles Guthaben · UC-10 Kaufen/Verkaufen · UC-11 Portfolio · UC-12 Live-Kurse · UC-13 Watchlist · UC-14 Responsive GUI.